SERVICENOW SYSTEM ADMINISTRATOR · NAAN MUDHALVAN (SMARTBRIDGE)

Script-Controlled ACL - Project Documentation

Phase 7 · Project Documentation

Deepak P (Team Lead) · Hemanthan B N · Ganesh A K · Irfan Rifayil Z

This is the consolidated project document for the ServiceNow System Administrator capstone "Script-Controlled ACL - Restrict Record Access Based on Field Value". It records the problem, objective, scope, design, modules, implementation, and verified results of a record-level access-control build delivered on the Personal Developer Instance dev443713.service-now.com (Australia release, 2026-09-28) by Team Lead Deepak P with Hemanthan B N, Ganesh A K, and Irfan Rifayil Z.

Introduction

An institution manages student "Institution Details" records inside ServiceNow. Out of the box, any user who can reach the table can read and edit every record, which breaks data privacy and record integrity across branches. This project implements record-level, field-value-driven access control so that what a user can see and do depends on both their role and the Branch field value on each record. It is a genuine hands-on configuration project built entirely on the native Now Platform - no external app, framework, or database is involved.

Objective

- Restrict record visibility and editing on the Institution Details table by branch.
- Drive access from a combination of user role (bb1-bb4) and the record's Branch field value.
- Demonstrate ServiceNow's Access Control List engine, including Advanced (scripted) ACLs using gs.hasRole.
- Prove the outcome objectively through impersonation of three distinct personas.

Scope

- In scope: one custom table, four roles, one test user, and four record-level ACLs (read/create/write/delete), with a scripted and conditioned READ rule.
- In scope: security_admin elevation, Background Scripts for repeatable setup, and impersonation-based verification.
- Out of scope: external integrations, custom UI, workflow/flow automation, and any table other than u_institution_details.
- Platform: a free ServiceNow Personal Developer Instance; the pattern is portable to any Now Platform instance.

Existing System vs Proposed System

Aspect	Existing (out of the box)	Proposed (this project)
Record visibility	Any user reaching the table sees all records	Branch users see only their own branch's records
Access basis	Table-level reach only	Role AND Branch field value, per operation
Read control	Uniform for all who can open the table	bb1 role + condition u_branch=EEE + gs.hasRole script

Aspect	Existing (out of the box)	Proposed (this project)
Data privacy	Cross-branch data exposed	Cross-branch data isolated
Auditability	Hard to reason about	Four explicit ACLs, one per operation

Module 1 - Users and Roles

Four operation-specific roles were created and all four granted to a single test user, "EEE User", so one login can exercise every path.

Role	Grants	Held by
bb1	Read	EEE User
bb2	Create	EEE User
bb3	Write (update)	EEE User
bb4	Delete	EEE User

Module 2 - Table and Fields

A custom table u_institution_details was created. The Branch choice field is the field-value that access control keys on. Four sample records span all three branches: ECE001 (ECE), EEE001 (EEE), EEE002 (EEE), CSE001 (CSE).

Column	Type
Student Roll Number	String
Student Name	Reference (User)
Faculty Name	Reference (User)
Branch	Choice (ECE / EEE / CSE)
Email	String
Phone Number	String
Description	String (4000)

Module 3 - Access Control Rules

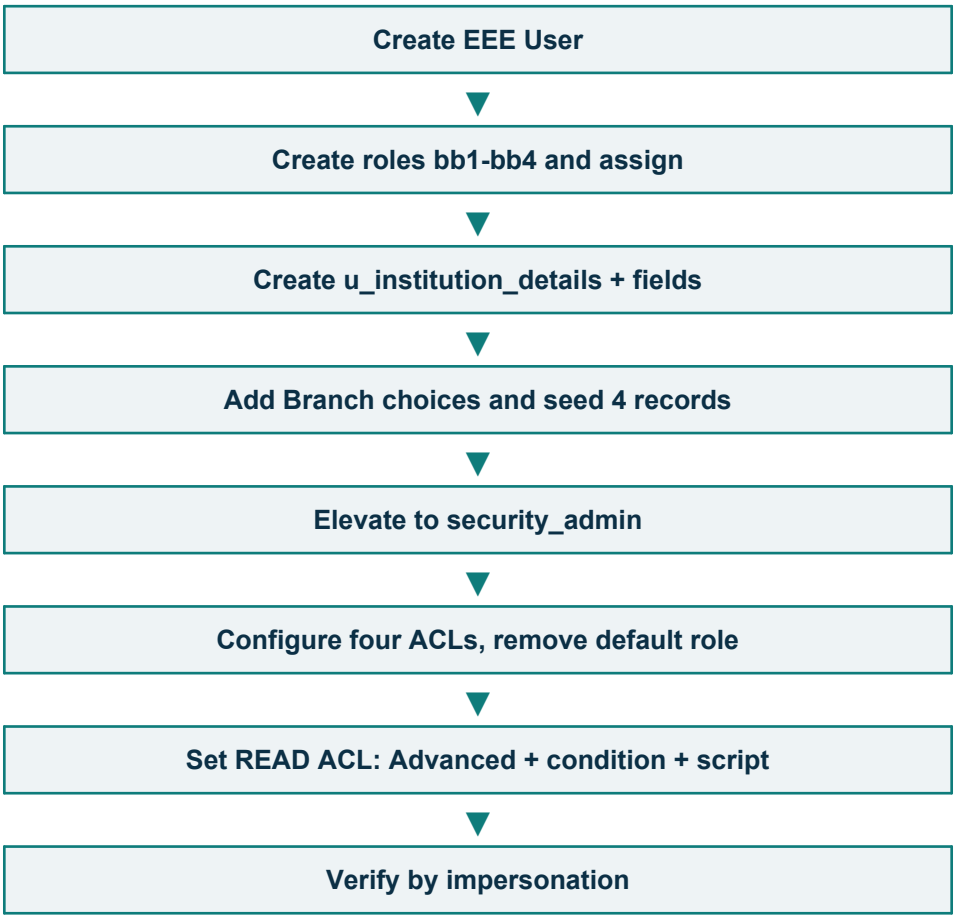
Creating the table auto-generated default ACLs carrying a role u_institution_details_user; that role was removed so each rule needs only its dedicated bb role. Editing ACLs required elevating to security_admin. Four record-level ACLs were then configured, one per operation.

Operation	Role	Type	Condition	Script
Read	bb1	Advanced	Branch is EEE (u_branch=EEE)	admin -> true, bb1 -> true, else false
Create	bb2	Default	None	None
Write	bb3	Default	None	None
Delete	bb4	Default	None	None

Module 4 - Verification

Verification was performed by impersonating three personas on 2026-09-28 and observing the live list and form behaviour, confirming the field-value-driven restriction works end to end.

Implementation Summary



Results

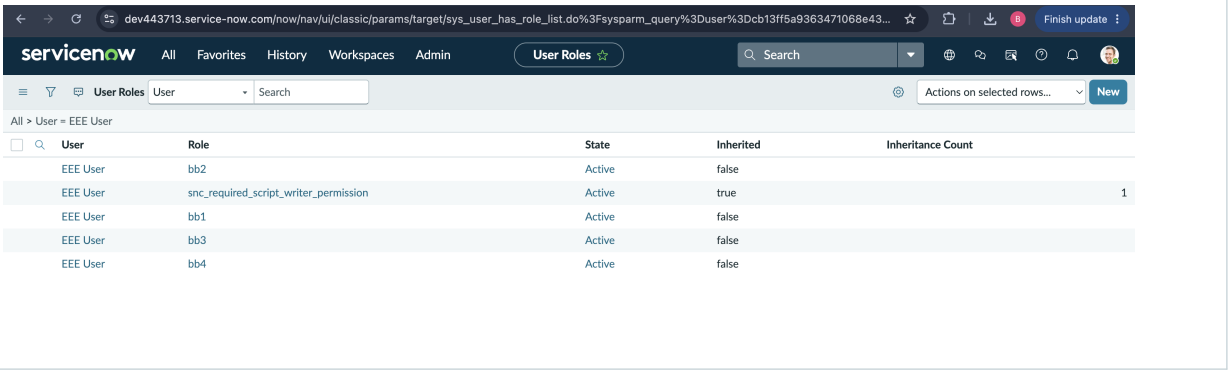
Persona	Roles	Observed result
System Administrator	admin	List shows all 4 records
EEE User	bb1, bb2, bb3, bb4	List shows only EEE001 and EEE002 (2 of 2); New button visible; EEE001 opens editable with Update and Delete
Abel Tuter	none of bb1-bb4	List replaced by "Security constraints prevent access to requested page"

Conclusion

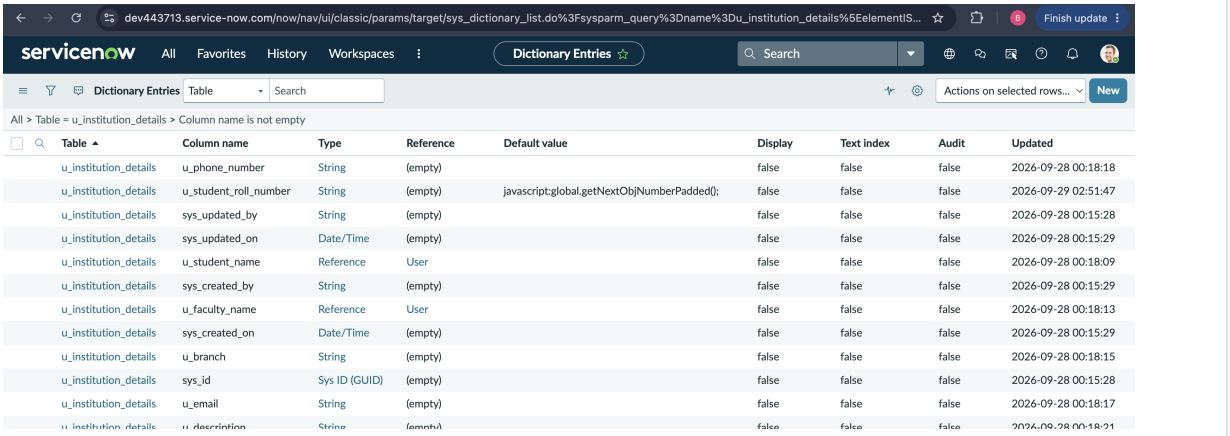
The project demonstrates record-level, field-value-driven access control on the native ServiceNow platform. By combining operation-specific roles (bb1-bb4) with an Advanced READ ACL that pairs a data condition (u_branch=EEE) with a gs.hasRole script, branch users are confined to their own records while administrators retain full visibility and unauthorised users are blocked entirely. The impersonation results confirm data privacy and record integrity are enforced as designed, using only standard Now

Platform ACL security, GlideScript, and Background Scripts - a repeatable pattern that generalises to any branch or field value.

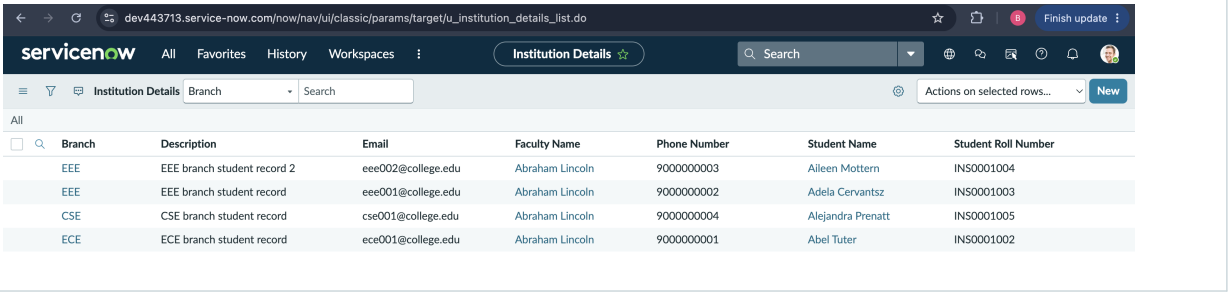
Evidence (from the ServiceNow instance)



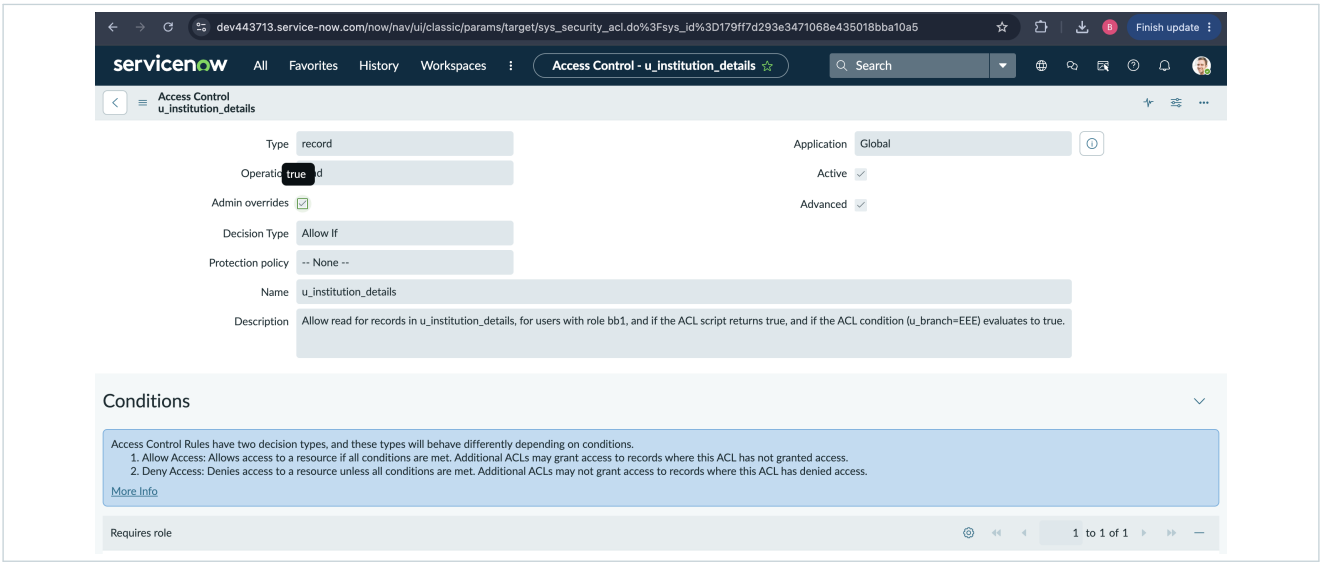
EEE User holding roles bb1–bb4



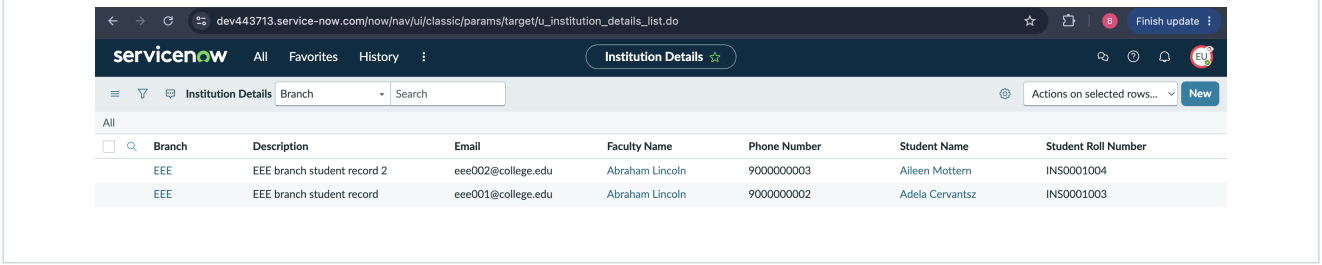
u_institution_details columns



Four sample records (admin view)



READ ACL: role bb1, condition and script



EEE User sees only the two EEE records